Reinforcement learning from human feedback

SESSION 2 · PREFERENCES AND REWARD MODELS

Pegah Alizadeh

pegahani.github.io

pegah.alizadeh@univ-lyon2.fr

Master's course, 2026–2027

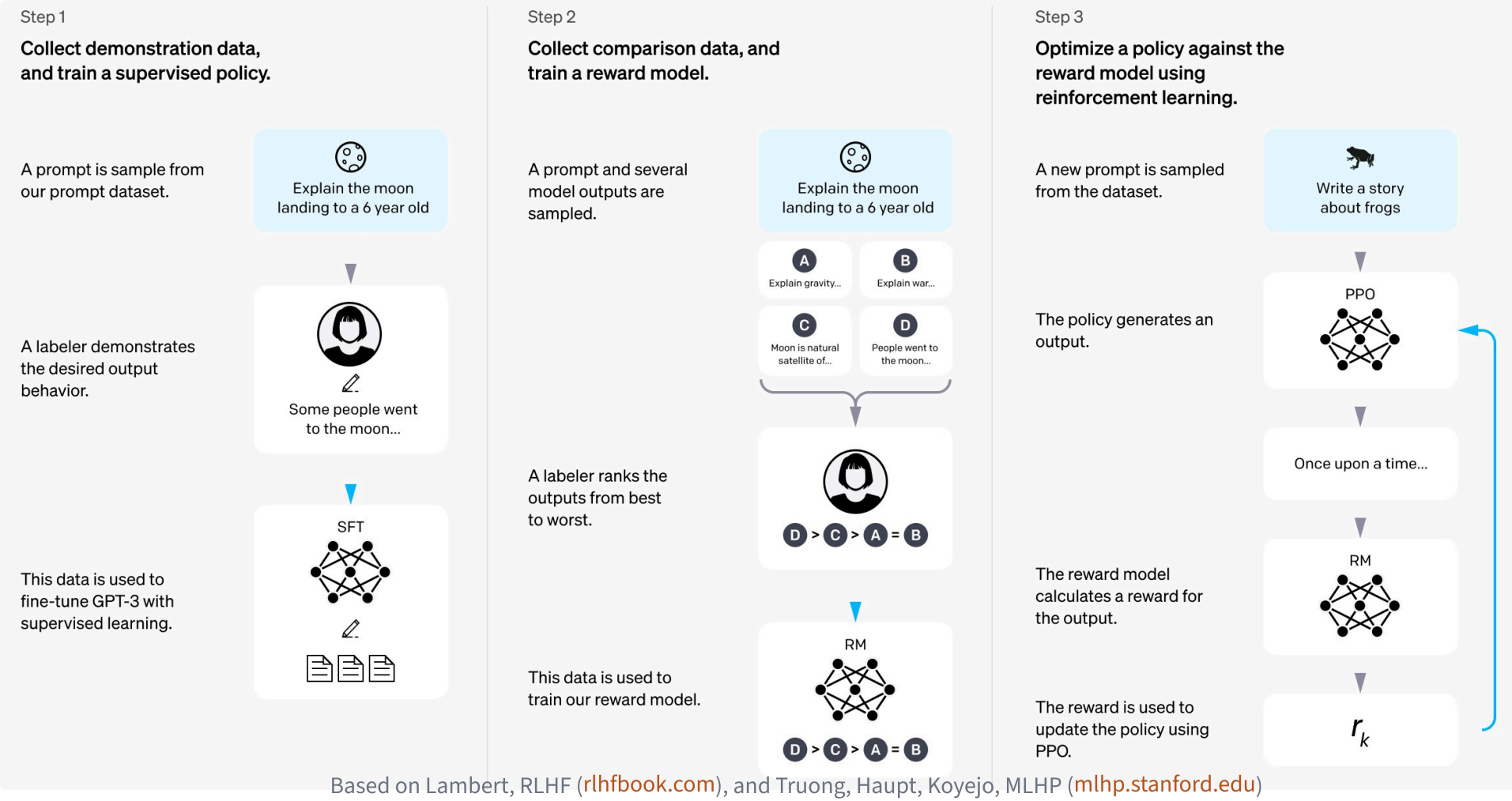
Last session

- **Language models in practice:** tokens, logits and log-probabilities; training is maximum likelihood
- **Instruction tuning (SFT):** the same loss, on the response only, thanks to the completion mask. The model learns to answer and to stop.
- **The lab:** a base model became an assistant, and we measured the change with an interval for the stop rate
- **Comparison data:** people compare responses, and we model their choices as random

Today: the next step of the recipe, learning a **reward model** from comparisons.

Where we are: InstructGPT's three steps

Session 1 covered step 1. Today: step 2, and the start of step 3. Source: Lambert, Ch. 1.



Ouyang et al., 2022

Today

1. **Why preferences, and why a number?** From comparisons to a scalar reward
2. **From comparisons to a reward model:** InstructGPT's steps 2 and 3
3. **Models of comparison:** where Bradley-Terry comes from, and when it fails
4. **Reward models for language models:** architecture, variants and judges
5. **Preference data:** how it's collected, binarized, and biased
6. **Lab:** train a reward model for a language model, and test what it learned

Today's main sources: Lambert, Chapters 5, 10 and 11; Truong et al., Chapters 1 and 2.

Why preferences, and why a number?

Instruction tuning and verifiable rewards need a notion of **correct**. Many things we want — which poem is better, which tone is kinder — have **no correct answer**, so we ask people to **compare**: cheap, and more reliable than asking for an absolute score.

Von Neumann and Morgenstern (1947): if preferences are complete, transitive, continuous and independent, they reduce to a single **utility function** — the result RLHF leans on to fit **one scalar reward** from comparisons.

In RLHF these conditions rarely hold exactly: preferences **drift**, depend on **framing**, can be **intransitive**, and are **multidimensional**. We build the reward model anyway, and come back to where this breaks.

Source: Lambert, Ch. 10, When preference replaces correctness; Lambert, Ch. 10, From utility to reward functions.

Von Neumann & Morgenstern, 1947

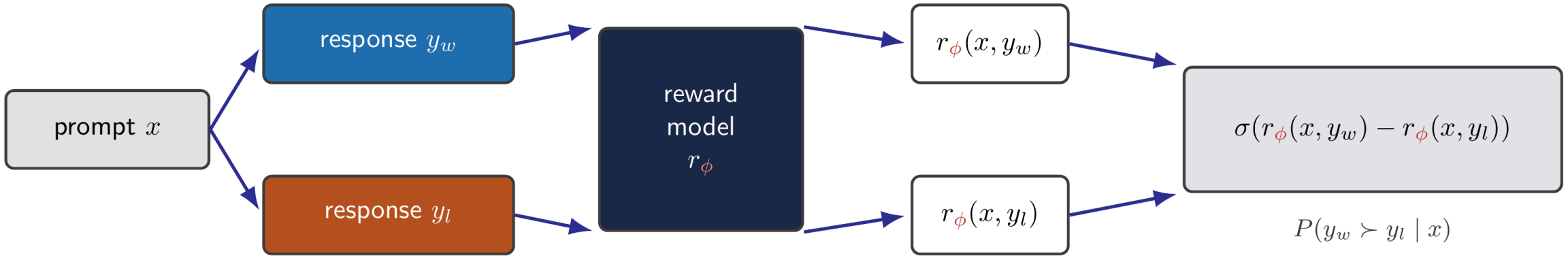
From comparisons to a reward model

Step 2: reward modeling

The data: comparisons $\mathcal{D} = \{(x, y_w, y_l)\}$, where people preferred the winner y_w to the loser y_l for the same prompt x.

The model: an **unknown** reward $r_\phi(x, y)$, one score per response: a transformer whose head outputs one number. **Red** marks what we **learn** from data, here the parameters ϕ.

The question: given two scores, how do we turn them into a probability that one response wins?



One prompt, two responses, two scores — the open question is how to turn the score gap into a probability.

Christiano et al., 2017; Ouyang et al., 2022

Where does Bradley-Terry come from?

A **probability model** is a mathematical form we assume matches how real judgments work, then fit to data. The classic choice, from 1952: **Bradley-Terry.** Each response has a latent strength $p_y > 0$, and

$$P(y_w \succ y_l \mid x) = \frac{p_{y_w}}{p_{y_w} + p_{y_l}}.$$

Reparametrize with $p_y = e^{r_\phi(x,y)}$: unbounded scores $r_\phi \in \mathbb{R}$ — exactly what a neural network head naturally outputs:

$$P(y_w \succ y_l \mid x) = \frac{e^{r_\phi(x,y_w)}}{e^{r_\phi(x,y_w)} + e^{r_\phi(x,y_l)}} = \sigma\left(r_\phi(x, y_w) - r_\phi(x, y_l)\right).$$

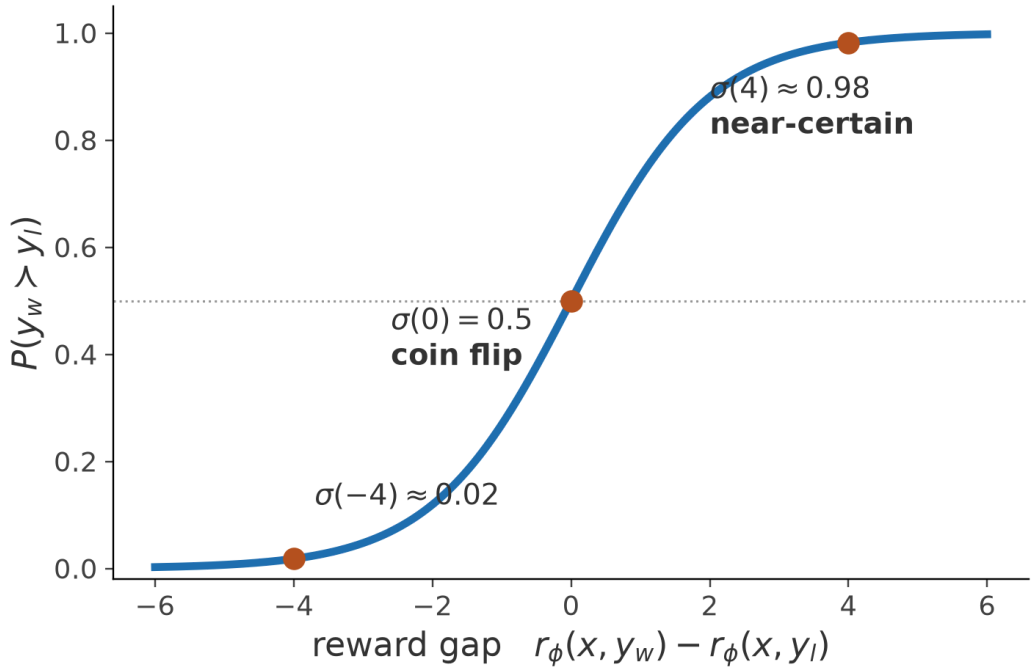
This is not the only possible model of preferences — but it's what InstructGPT used, and it stuck. A useful approximation, not a law of nature.

Source: Lambert, Ch. 5, Training a Bradley-Terry reward model.

Bradley & Terry, 1952

The shape of σ

A reward gap of 0 is a coin flip; a large gap is near-certain — and it saturates, so once the model is confident, pushing the gap further barely changes the probability.



The sigmoid turns a reward gap into a win probability: flat near 0, saturating at the extremes.

From the probability to a loss

We want ϕ that makes the model assign **high probability to the preferred response**. Start from maximizing that probability directly:

$$\phi^* = \arg\max_{\phi} \frac{e^{r_\phi(x,y_w)}}{e^{r_\phi(x,y_w)} + e^{r_\phi(x,y_l)}}$$

Deriving the reward model loss

$$\phi^* = \arg\max_{\phi} \frac{e^{r_\phi(x,y_w)}}{e^{r_\phi(x,y_w)} + e^{r_\phi(x,y_l)}}$$

Divide numerator and denominator by $e^{r_\phi(x,y_w)}$:

$$= \arg\max_{\phi} \frac{e^{r_\phi(x,y_w)} / e^{r_\phi(x,y_w)}}{\left(e^{r_\phi(x,y_w)} + e^{r_\phi(x,y_l)}\right) / e^{r_\phi(x,y_w)}}$$

Deriving the reward model loss

$$\phi^* = \arg\max_{\phi} \frac{e^{r_\phi(x,y_w)} / e^{r_\phi(x,y_w)}}{\left(e^{r_\phi(x,y_w)} + e^{r_\phi(x,y_l)}\right) / e^{r_\phi(x,y_w)}}$$

The numerator becomes 1; in the denominator, $e^{r_\phi(x,y_l)} / e^{r_\phi(x,y_w)} = e^{r_\phi(x,y_l) - r_\phi(x,y_w)}$:

$$= \arg\max_{\phi} \frac{1}{1 + e^{r_\phi(x,y_l) - r_\phi(x,y_w)}}$$

Deriving the reward model loss

$$\phi^* = \arg\max_{\phi} \frac{1}{1 + e^{r_\phi(x,y_l) - r_\phi(x,y_w)}}$$

Rewrite the exponent as $-\left(r_\phi(x, y_w) - r_\phi(x, y_l)\right)$ and recognize the sigmoid $\sigma(z) = \frac{1}{1+e^{-z}}$:

$$= \arg\max_{\phi} \sigma\left(r_\phi(x, y_w) - r_\phi(x, y_l)\right)$$

A large reward gap means a near-certain choice; a gap of 0 means a coin flip.

Deriving the reward model loss

$$\phi^* = \arg\max_{\phi} \; \sigma\left(r_\phi(x, y_w) - r_\phi(x, y_l)\right)$$

Training maximizes the **log**-likelihood over the dataset, not the raw probability (take $\log$ *before* averaging, since $\mathbb{E}[\log P] \neq \log \mathbb{E}[P]$), then flip the sign to get a loss to minimize:

$$\mathcal{L}_{\text{RM}}(\phi) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left[\log \sigma\left(r_\phi(x, y_w) - r_\phi(x, y_l)\right)\right]$$

This model is called Bradley-Terry, and it's the subject of today's session.

Source: Lambert, Ch. 5, Training a Bradley-Terry reward model; Truong et al., §1.6.2 Connecting Rasch to Bradley-Terry.

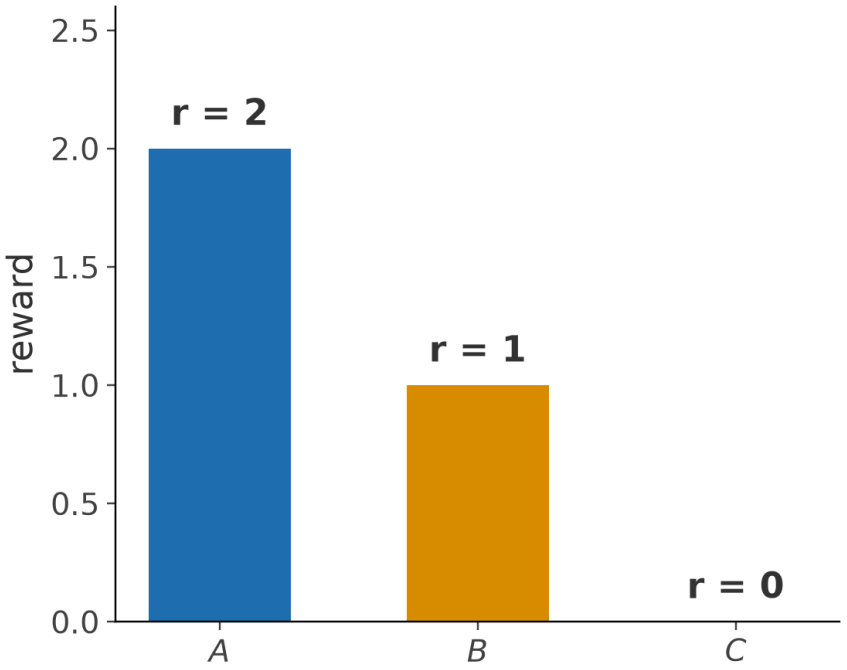
A worked example

Three responses with rewards $r_A = 2, r_B = 1, r_C = 0$, and $P(j \succ k) = \sigma(r_j - r_k)$:

Comparison	Reward difference	Probability
$A \succ B$	1	$\sigma(1) \approx 0.731$
$A \succ C$	2	$\sigma(2) \approx 0.881$
$B \succ C$	1	$\sigma(1) \approx 0.731$

Only reward **differences** matter: adding a constant to every reward changes nothing.

Source: Truong et al., §1.2 A running example.



Truong et al., 2025

From pairs to rankings: Plackett-Luce OPTIONAL

Labelers often **rank** several responses (InstructGPT: 4 to 9). **Plackett-Luce** fills a ranking **one place at a time**, like drawing names from a hat: each draw is a softmax over the responses still left.

Example: $r_A = 2, r_B = 1, r_C = 0$. How likely is the ranking $A \succ B \succ C$?

Place	Still left	Probability of this pick
1st: A	A, B, C	$e^2/(e^2 + e^1 + e^0) \approx 0.665$
2nd: B	B, C	$e^1/(e^1 + e^0) = \sigma(r_B - r_C) \approx 0.731$
3rd: C	C	1
Ranking		**Product:** ≈ 0.486

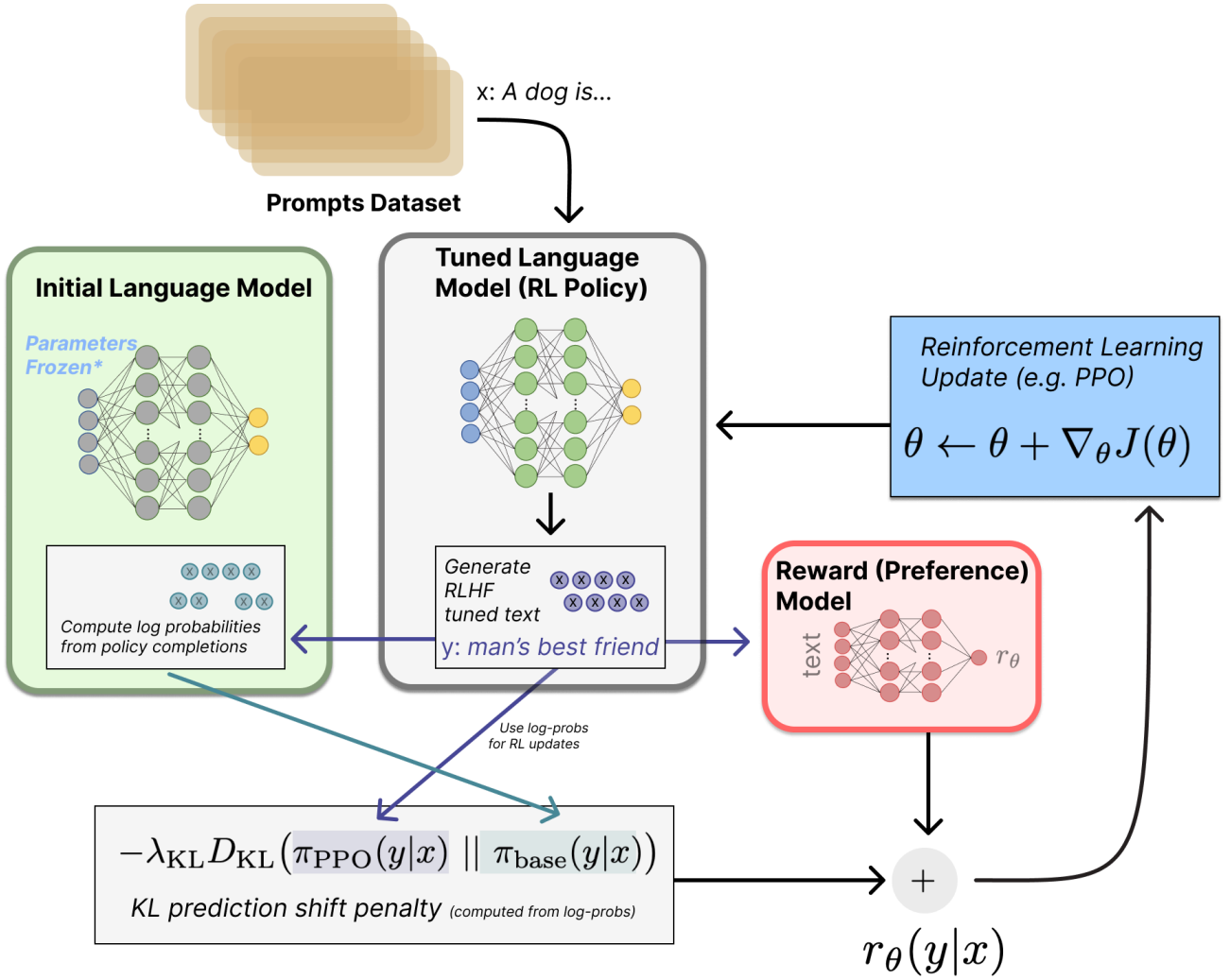
With only two responses left, each draw is exactly Bradley-Terry (the 2nd-place row above): $e^{r_A}/(e^{r_A} + e^{r_B}) = \sigma(r_A - r_B)$, where $\sigma(z) = 1/(1 + e^{-z})$ is the sigmoid.

Luce, 1959; Plackett, 1975

RL against the reward model: a general view

r_ϕ is already trained and fixed (step 2); RL only updates π_θ.

- Sample prompts x_i
- Generate $y_i \sim \pi_\theta(\cdot \mid x_i)$
- Score with $r_\phi(x_i, y_i)$
- Update the policy by policy gradient — **a choice:** InstructGPT used PPO; others work too
- Often a **KL penalty** to stay close to the reference model — also a choice, not required



Ouyang et al., 2022

Three ways to use preferences OPTIONAL

Three common ways to move the model toward the preferred answers:

	Best-of-n, then SFT (Supervised Fine-Tuning)	Online RL (PPO, Proximal Policy Optimization)	DPO (Direct Preference Optimization)
How it works	Generate n answers, keep the one the reward model likes best, fine-tune on it	InstructGPT's step 3: generate, score, update, repeat	A loss on the preferred and rejected answers directly
Reward model?	Yes	Yes	No
RL loop?	No: plain SFT	Yes	No: plain gradient descent
Cost	Low	High	Low

Same goal: higher reward, without drifting far from the starting model.

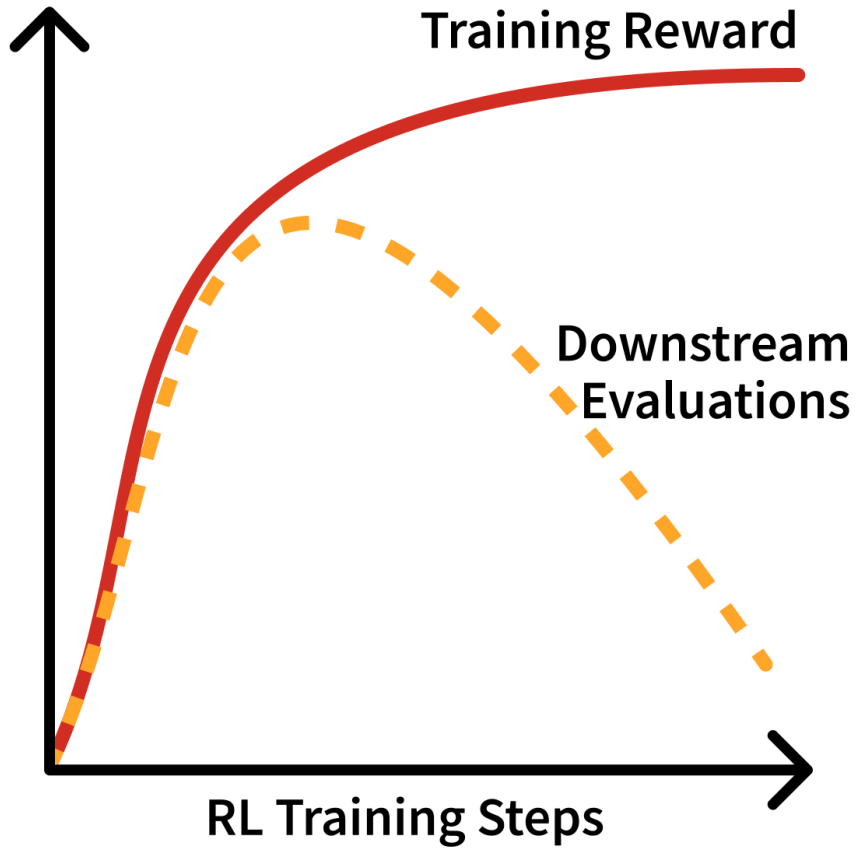
Source: Lambert, Ch. 3, Optimization tools.

Watch out: the reward is only a proxy

A reward model is only *correlated* with what people want. **Goodhart's law:** "When a measure becomes a target, it ceases to be a good measure."

- **Reward hacking:** the score climbs, real quality falls
- **Verbosity:** longer answers score higher
- **Sycophancy:** telling users what they want to hear
- **Over-refusal:** refusing reasonable requests

Source: Lambert, Ch. 14, Managing proxy objectives.



Training reward keeps climbing while downstream quality peaks and falls (Lambert, Ch. 14).

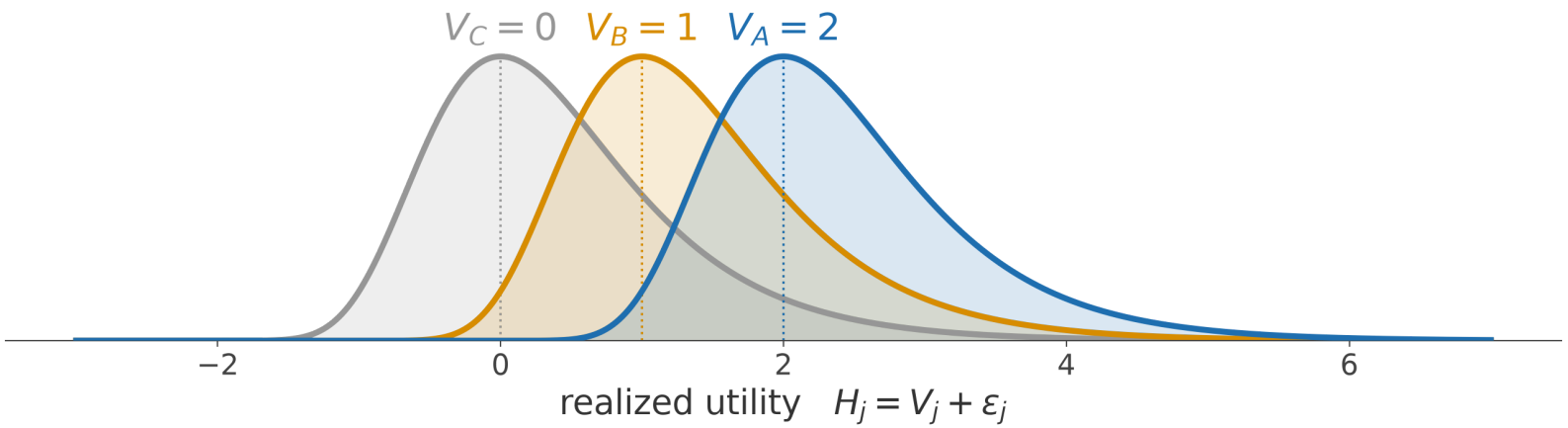
Models of comparison

Pivots from how to train/use r_ϕ to why Bradley-Terry specifically is the right model.

Why a reward model needs noise

If the bigger V_j always won, every comparison would be 1 or 0, never 0.73. Real annotators disagree, so each response gets a **random utility** $H_j = V_j + \varepsilon_j$, and the highest realized H_j wins that round. V_j is what we learn; ε_j is everything we don't model (mood, attention, framing).

The noise distribution determines the model. The usual choice is **Gumbel noise**: a fast rise, then a long tail. In the figure, the horizontal axis is the value H_j can take; each curve peaks at its response's V_j, and its width is the noise.



From Gumbel noise to Bradley-Terry

With i.i.d. Gumbel noise,

$$p(H_j > H_k) = \sigma(V_j - V_k),$$

exactly Bradley-Terry. This is the **Gumbel-max trick** from machine learning: add Gumbel noise to K scores and take the argmax, and item j wins with probability $e^{V_j} / \sum_k e^{V_k}$, the softmax. With two items, the softmax is $\sigma(V_j - V_k)$.

So Bradley-Terry is derived, not assumed. Another noise gives another model: Gaussian noise gives *probit*, with Φ in place of σ.

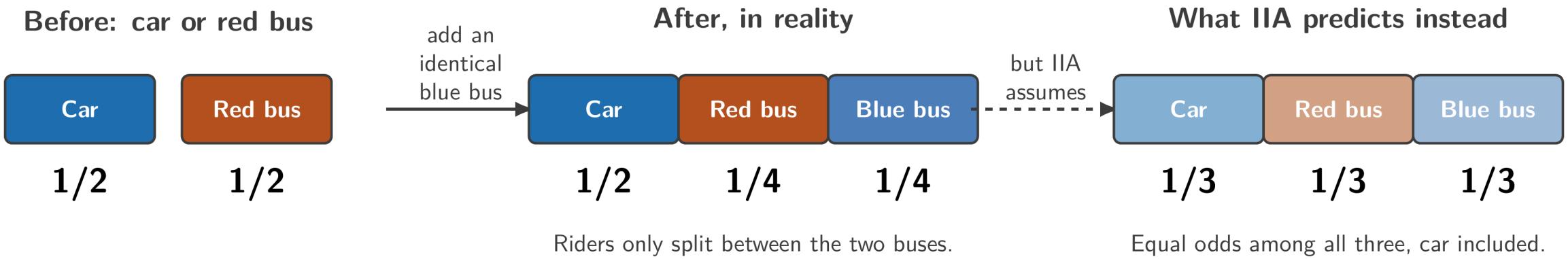
Source: Truong et al., §1.7.

Truong et al., 2025

The price of Gumbel noise: IIA

Gumbel noise is equivalent to **independence of irrelevant alternatives (IIA)**: adding a third option never changes the odds between two others. That fails in two ways:

- **Similar options:** two near-duplicate responses split their "votes", like a red and a blue bus
- **Different people:** a mixture of annotator groups is not one Bradley-Terry model



Source: Truong et al., §1.8 IIA; §1.11 Beyond Bradley-Terry.

Truong et al., 2025

Only differences are identified

Adding the same constant to every utility changes nothing:

$$\sigma\left((V_j + c) - (V_k + c)\right) = \sigma(V_j - V_k).$$

- **Anchor one parameter,** for example $V_1 = 0$.
- **Fitting is a logistic regression:** each comparison is a row $x = e_j - e_k$ and $p(j \succ k) = \sigma(x^\top V)$. You fit it, with Fisher-information intervals, in **HW1**.

Source: Truong et al., §1.9 Identification; §2.3 Maximum likelihood.

Truong et al., 2025

Reward models for language models

Reward model architecture

A language model with a **linear head** that outputs **one scalar**, read at the **last token**. It's the model you build in today's lab.

```
class BradleyTerryRewardModel(nn.Module):
    def __init__(self, base_lm):
        super().__init__()
        self.lm = base_lm
        self.head = nn.Linear(self.lm.config.hidden_size, 1)

    def forward(self, input_ids, attention_mask):
        hidden = self.lm(input_ids=input_ids, attention_mask=attention_mask,
                         output_hidden_states=True).hidden_states[-1]
        last = attention_mask.sum(dim=1) - 1          # index of the last real token
        batch = torch.arange(hidden.size(0))
        return self.head(hidden[batch, last]).squeeze(-1)   # one score per response
```

Source: Lambert, Ch. 5, Architecture.

Ouyang et al., 2022

The loss is simple

The Bradley-Terry loss we derived, in three lines:

```
rewards_chosen = model(**inputs_chosen)
rewards_rejected = model(**inputs_rejected)

loss = -nn.functional.logsigmoid(rewards_chosen - rewards_rejected).mean()
```

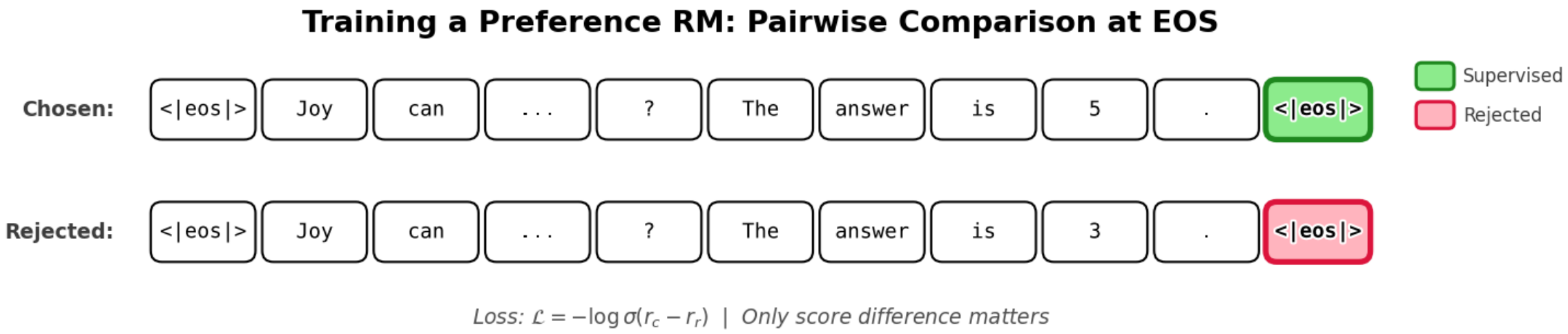
It's binary cross-entropy. The reward model learns to classify which of the two responses was preferred. Anthropic writes the same loss as $\log\left(1 + e^{r_\phi(x,y_l) - r_\phi(x,y_w)}\right)$, which is equal because $-\log\sigma(\Delta) = \log(1 + e^{-\Delta})$.

Practical note: reward models are usually trained for only **one epoch**, to avoid overfitting to the preference data.

Source: Lambert, Ch. 5, Implementation example.

Reward model training visualized

Each response gets **one score, at its last token** (the green and red boxes). The loss $-\log \sigma(r_w - r_l)$ pushes the **green** score up and the **red** score down.



Training a preference reward model: both responses are scored at their last token, and the loss depends only on the difference.

Ouyang et al., 2022

Why one score, at the last token?

- **Bradley-Terry needs one number per response,** but a transformer outputs one vector per token. So we put the head at **one** position.
- **The last token is the only one that has read everything.** Attention is causal: position t only sees the tokens up to t. Only the last token has seen the whole prompt and the whole response.
- **In the figure,** chosen and rejected are identical until “5” vs “3”. Any earlier position would give both responses the **same** score. Only the last token can tell them apart.

All other tokens get no score of their own: they only feed information into the last hidden state. Later today, ORMs do the opposite and put a score on **every** token.

Source: Lambert, Ch. 5, Architecture.

Reward model variants

Same Bradley-Terry idea, when the data says more than “A beats B”. In blue on each slide: the limitation it fixes.

Variants of the loss: margin loss

Why? Plain Bradley-Terry only knows **who** won, not **by how much**: a narrow preference and a landslide push the gradient equally hard.

When annotators give **Likert ratings** (for example 1 to 5), the size of the preference can inform training. Llama 2 added a margin $m(y_w, y_l)$:

$$\mathcal{L}(\phi) = -\log \sigma \left(r_\phi(x, y_w) - r_\phi(x, y_l) - m(y_w, y_l) \right).$$

Example: the chosen response is rated 5 and the rejected one 2, so $m = 3$. The model must now beat the rejected response by a **larger score gap** before the loss gets small.

Llama 3 removed the margin term: the team saw diminishing improvements at scale.

Source: Lambert, Ch. 5, Margin loss.

Touvron et al., 2023

Variants of the loss: several comparisons per prompt

Why? Labeling one pair at a time is expensive. Ranking K responses at once gives $\binom{K}{2}$ pairs for the price of one task. InstructGPT ranked 4 to 9 responses per prompt.

Example. A labeler ranks four responses: $A \succ B \succ C \succ D$. That one judgment gives $\binom{4}{2} = 6$ pairs:

$$A \succ B, \quad A \succ C, \quad A \succ D, \quad B \succ C, \quad B \succ D, \quad C \succ D.$$

The catch: these 6 pairs are not 6 independent labels. They come from one person, at one moment, with one reading of the prompt. If the labeler overrated A, the same mistake appears in **3 pairs at once** ($A \succ B, A \succ C, A \succ D$).

Source: Lambert, Ch. 5, Multiple comparisons.

Ouyang et al., 2022

Several comparisons per prompt: why it matters, and the fix

- **Overconfidence:** the pairs carry less information than their number suggests, so standard errors are too small.
- **Overfitting:** $K = 9$ gives 36 pairs, $K = 4$ only 6, so the model mostly learns the long rankings and their labelers' taste. InstructGPT saw this.

The fix: weight each prompt's pairs by $1/\binom{K}{2}$, so **one ranking counts as one observation** (and keep them in one batch):

$$\mathcal{L}(\phi) = -\frac{1}{\binom{K}{2}} \mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}} \left[\log \sigma \left(r_\phi(x, y_w) - r_\phi(x, y_l)\right)\right].$$

Same issue when one annotator gives many labels: you measure it in HW1. Source: Lambert, Ch. 5, Multiple comparisons.

Variants of the loss: K-wise loss

OPTIONAL

Why? Splitting a ranking into pairs creates correlated data, which we then have to reweight. Why not model the whole ranking directly?

Train on the full ranking of K responses with the **Plackett-Luce** model from earlier today: fill the ranking one place at a time, each place a softmax over the responses still left.

$$P(y_{(1)} \succ \cdots \succ y_{(K)} \mid x) = \prod_{k=1}^{K} \frac{e^{r_\phi(x, y_{(k)})}}{\sum_{j=k}^{K} e^{r_\phi(x, y_{(j)})}}.$$

Starling 7B and 34B were trained this way.

With $K = 2$, this is exactly Bradley-Terry. The loss we derived is the two-response case of this one.

Source: Lambert, Ch. 5, K-wise loss.

Zhu et al., 2023; Zhu et al., 2024

Other kinds of reward model

When a human preference between two answers is not the signal we have, or not the one we need.

Outcome reward models (ORMs)

Why? Bradley-Terry needs **pairs** and a human judgment. In math or code, we can simply **check** whether the answer is correct: one label per response, cheaper and more reliable.

An ORM learns from that label directly, with no pairs:

$$\mathcal{L}_{\mathrm{CE}}(\phi) = -\mathbb{E}_{(x,y,c)\sim\mathcal{D}} \frac{1}{|y|} \sum_{t=1}^{|y|} \Big[c \log p_t + (1-c) \log (1-p_t) \Big],$$

where $c \in \{0, 1\}$ says whether response y is correct, and $p_t = p_\phi(x, y_{\le t})$ is the predicted probability of correctness at token t, from the prompt and the response **up to token** t.

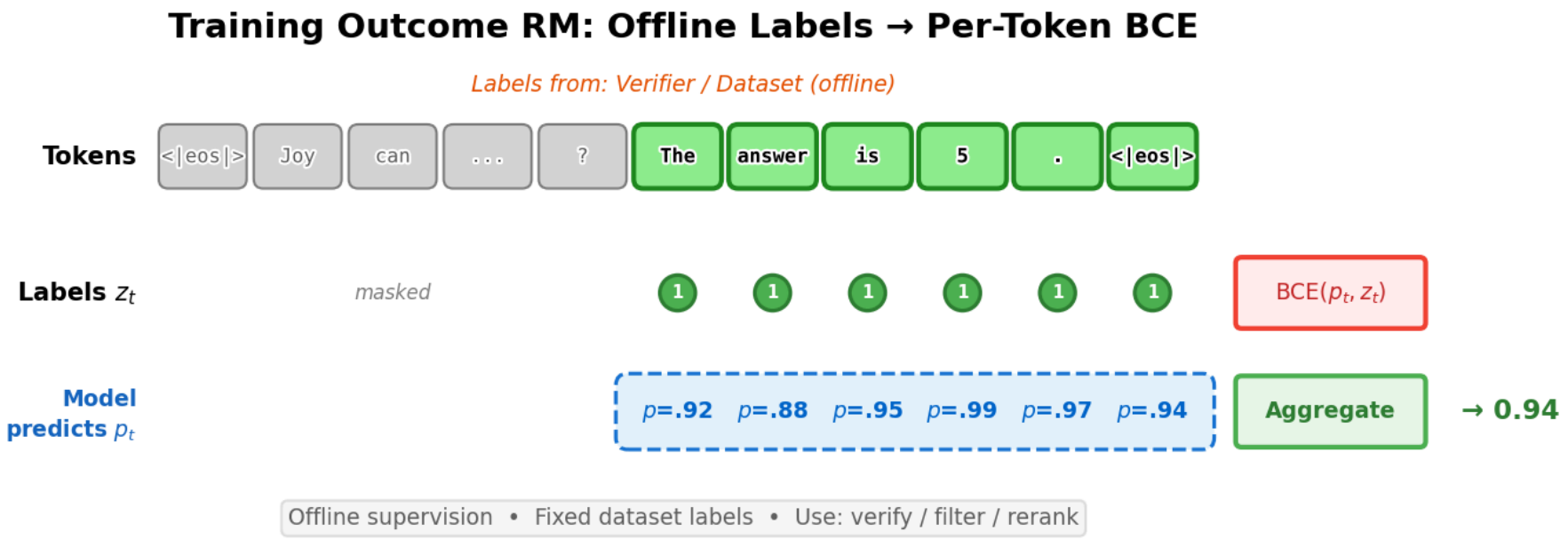
Unlike Bradley-Terry: one response and a label instead of a pair, and a probability **at every token** instead of one score at the last token.

Source: Lambert, Ch. 5, Outcome reward models.

Cobbe et al., 2021

ORM training visualized

The label $c \in \{0, 1\}$ is **copied to every response token**. At each token, the model predicts $p_t = p_\phi(x, y_{\leq t})$, the probability that the answer will be correct, trained with **BCE (binary cross-entropy)**, the logistic-regression loss: $\mathrm{BCE}(p_t, c) = -\big[c \log p_t + (1 - c) \log(1 - p_t)\big]$.



Same transformer, different heads

A transformer outputs **one hidden vector per token**. These models differ only in **which head** sits on top, and **where**:

Model	Head on each hidden vector	Used at	Target
Language model	Scores over the vocabulary	Every token	The next token
Preference RM	One number	The **last** token only	Chosen beats rejected (Bradley-Terry)
ORM	One number, then a sigmoid	**Every** response token	"The answer will be correct" (BCE)

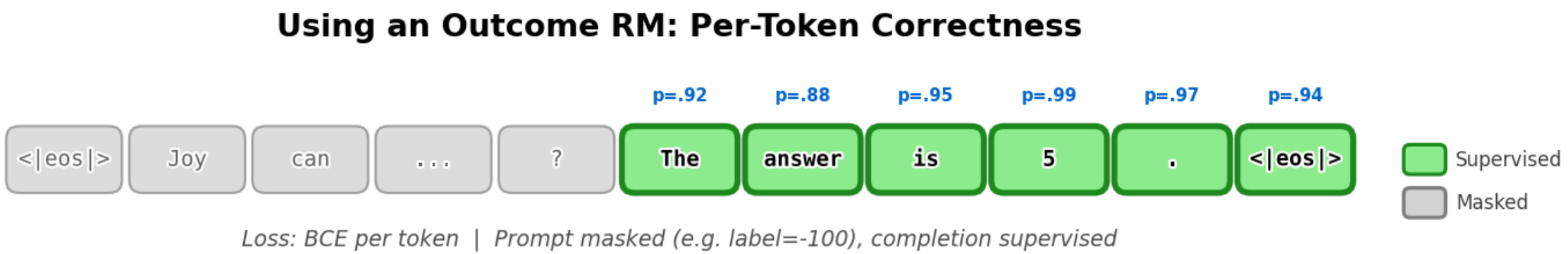
No one labels each token by hand: one label per response is copied to every token. Attention is causal, so p_t only sees the response **up to token** t: early tokens give uncertain predictions, late tokens sharper ones. Like pretraining, every token has a target; here it's the same one: will this answer be correct?

Source: Lambert, Ch. 5, Outcome reward models.

ORM inference

OPTIONAL

At inference, the ORM outputs a **probability of correctness at every token**. These can be combined into one score per response, to filter or rerank candidate answers.



At inference, an ORM outputs a probability of correctness at every response token.

Common confusion: an ORM is not Bradley-Terry on correct answers

OPTIONAL

Some papers say "outcome reward model" for a **Bradley-Terry model trained on (correct, incorrect) pairs**. These are two different models:

	Bradley-Terry on correct vs. incorrect	ORM
Training input	Two responses (a pair)	One response and a label
Output	One scalar at the last token	A probability at every token
Loss	$-\log \sigma\left(r_{\text{correct}} - r_{\text{incorrect}}\right)$	Binary cross-entropy at every token

A Bradley-Terry model trained on correctness pairs is still a **preference reward model**. It just uses correctness as the preference.

Source: Lambert, Ch. 5, Outcome reward models.

Process reward models (PRMs)

Why? An ORM only sees the final answer, so a lucky guess after wrong reasoning gets full credit, and a wrong answer doesn't tell us **where** it went wrong.

A PRM scores **each reasoning step**. The solution is cut into steps (at each newline), and each step gets one of **three labels**: -1 **wrong**, 0 **neutral** (acceptable but no real progress), $+1$ **correct**. The head predicts the label **at each step boundary**; other tokens are masked:

$$\mathcal{L}_{\text{PRM}}(\phi) = -\mathbb{E}_{(x,s)\sim\mathcal{D}}\left[\sum_{i=1}^{K} \log p_\phi(z_i \mid x, s_{\le i})\right],$$

where $s_{\le i}$ is the solution up to the end of step i, and $z_i \in \{-1, 0, +1\}$ is its label.

Source: Lambert, Ch. 5, Process reward models.

Lightman et al., 2024

PRM labels: a worked example

Problem: Joy reads 8 pages in 20 minutes. How many hours for 120 pages? (Answer: 5.)

Step	A model's solution	Label
1	"First, find her reading rate."	0: true, but no progress
2	"8 / 20 = 0.4 pages per minute."	$+1$: correct
3	"120 × 0.4 = 48 minutes."	-1: **wrong**, should be 120 / 0.4 = 300
4	"So it takes 48 / 60 = 0.8 hours."	-1: wrong, carries the error forward

- **An ORM** sees one label for the whole solution: $c = 0$, "wrong". It can't say where.
- **A PRM** points to **step 3**, and gives credit to steps 1 and 2.

Step labels are expensive: PRM800K has about 800,000 of them, written by people, for math. Source: Lambert, Ch. 5, Process reward models; Lightman et al., 2023.

Comparing reward model types

Model	Predicts	Trained on	Output
Preference RM	Quality of the whole response	Chosen and rejected pairs	One scalar, at the last token
ORM	Whether the answer is correct	Correct/incorrect labels	A probability at every token
PRM	Whether each step is correct	Step-level labels	A score at each step boundary

Which one to use?

- **Preference RM** when only people can judge: helpfulness, tone, style
- **ORM** when the final answer can be checked: math, code
- **PRM** when you can also afford labels for every step, and want to know **where** reasoning fails

Source: Lambert, Ch. 5, Comparison.

Language models as judges

Why? Training a reward model needs thousands of labeled pairs and a GPU. A strong language model can judge out of the box.

So **prompt it** to compare two responses (Zheng et al., 2023):

> *Please act as an impartial judge and evaluate the quality of the responses provided by two AI assistants… output "[[A]]" if assistant A is better, "[[B]]" if assistant B is better, and "[[C]]" for a tie.*

- **Cheap to set up,** and widely used for evaluation and for creating training data. It spawned benchmarks such as MT-Bench, AlpacaEval and Arena-Hard.
- **Usually weaker** than trained reward models on reward model benchmarks (Lambert et al., 2025)
- **Has its own biases,** such as favoring the first response shown, longer answers or its own style

Source: Lambert, Ch. 5, LLM-as-a-judge.

Evaluating reward models OPTIONAL

How do we know a reward model is good? **Test it on held-out preference pairs** and measure how often it ranks the chosen response above the rejected one. That's what you'll do in the lab.

Public benchmarks have grown quickly since the first one, RewardBench:

- **General chat:** RewardBench, RewardBench 2, RMB, RM-Bench
- **Specialized:** M-RewardBench (multilingual), RAG-RewardBench, RewardMATH
- **Process reward models:** PRM Bench, ProcessBench
- **Agents and multimodal:** Agent-RewardBench, Multimodal RewardBench, VL RewardBench

Source: Lambert, Ch. 5, Further reading.

Lambert et al., 2025

Preference data

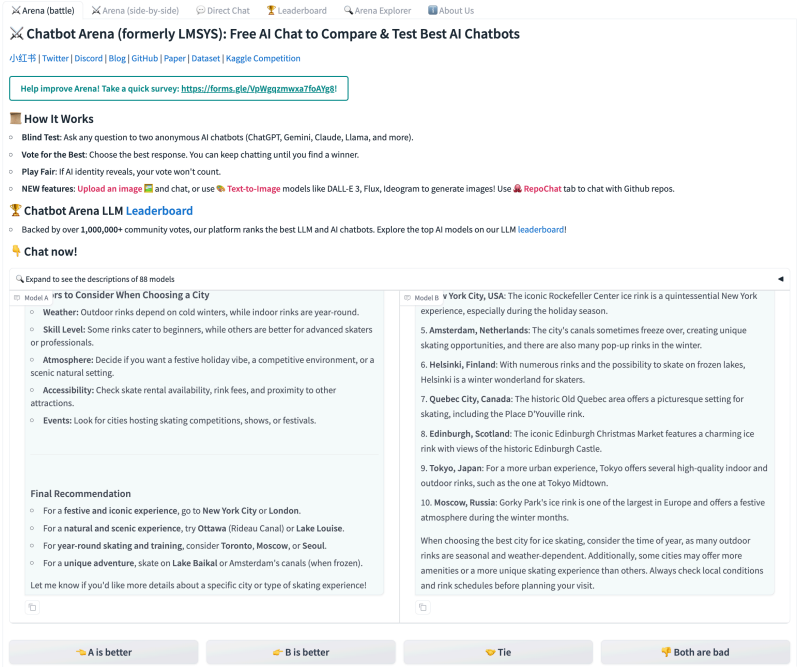
How preference data is collected

Interfaces: pairwise with ties, graded scales, thumbs up or down, pick one of many. **Each interface shapes the preference it captures.**

Who labels:

- **Human annotators,** through vendors: expensive and slow
- **Product users:** A/B tests, thumbs up or down
- **AI feedback:** a strong model labels the pairs. **UltraFeedback, your lab's dataset,** is labeled by GPT-4.
- **Structured pairs:** where answers can be checked, correct beats incorrect

Source: Lambert, Ch. 11, Interfaces; Sourcing.



Pairwise comparison with a tie option, in an early version of Chatbot Arena.

Bai et al., 2022

Ratings, Likert scales, and binarizing

- **Ratings:** a score for one response on its own, such as 4 out of 5
- **Rankings:** a relative judgment between responses. Almost everyone trains on these.

A **Likert scale** records a graded comparison:

1	2	3	4	5
A much better	A better	Tie	B better	**B much better**

For Bradley-Terry, graded answers are **binarized** into chosen and rejected. The strength of the preference is thrown away, or kept in a margin loss.

In the lab: UltraFeedback rates each response, and the highest- and a lower-rated one become your chosen/rejected pair. You'll check whether pairs with a bigger rating gap are easier to predict.

Source: Lambert, Ch. 11, Rankings vs. ratings.

Biases in the data

Systematic biases in annotation pass into the reward model, and then into the policy:

- **Verbosity:** longer answers are rated higher (Singhal et al., 2023)
- **Formatting:** lists and bold text look better (Zhang et al., 2025)
- **Sycophancy:** agreeing with the user is preferred to being right (Sharma et al., 2024)
- **Prefix bias:** the opening of a response drives the label (Kumar et al., 2025)

This is where the proxy problems from earlier today come from. Random label flips mostly cost efficiency; a systematic bias is learned as a genuine preference. More data doesn't remove it, and optimization amplifies it.

And whose preferences? Annotators disagree, and no aggregation rule satisfies every fairness criterion (Arrow, 1950).

Source: Lambert, Ch. 11, Bias; Open questions.

Lab: a reward model for a language model

Lab: what you'll do

We turn **Qwen3-0.6B** into a reward model and train it on **UltraFeedback**: real responses, with preferences from GPT-4.

1. **Look at the data:** is the chosen response better, or just longer?
2. **Build the model:** a linear head on the last token's hidden state, and the Bradley-Terry loss
3. **Train** on 3,000 pairs, in about 15 minutes on a free T4
4. **Evaluate it like a statistician:** held-out accuracy with an interval, a "longer wins" baseline, rating gaps, calibration, and length bias
5. **Try to fool it** with your own responses

Open the lab in Colab (or import the `.ipynb` from github.com/pegahani/rlhf-course into Kaggle) and choose a **T4 GPU**. Use **File → Save a copy in Drive** to keep your work.

Source: Lambert, Ch. 5, Suggested experiments; Implementation example.

Key points

- **A reward model** turns comparisons into one score per response, $r_\phi(x, y)$, then trains the policy
- **Bradley-Terry:** $P(y_w \succ y_l) = \sigma(r_w - r_l)$. Its loss, $-\log \sigma(r_w - r_l)$, is binary cross-entropy
- **Where it comes from:** Gumbel noise on utilities. The price is IIA, which fails for similar options and diverse annotators
- **Only differences are identified,** and fitting is a logistic regression (HW1)
- **For a language model:** a linear head on the **last token's** hidden state
- **Other reward models:** ORMs check answers at every token, PRMs check every step, language models can judge without training
- **The reward is a proxy:** biases in the data (length, format, sycophancy) are learned as real preferences

Before next session

Homework (not graded): HW1, the Bradley-Terry model from scratch: maximum likelihood, Fisher information and the coverage of intervals. Open it in Colab, or find `hw1-bradley-terry.ipynb` at github.com/pegahani/rlhf-course. No GPU needed. Finish HW0 first if you haven't yet.

Reading, to review today's session

- **Lambert, *RLHF*:** Chapter 5 (reward models) and Chapter 11 (preference data)
- ***Machine Learning from Human Preferences*:** Chapter 1 (models of comparison)

References (1/3)

Arrow, K. "*A difficulty in the concept of social welfare*." *Journal of political economy*, 1950.

Bai, Y., Jones, A., Ndousse, K., Askell, A., Chen, A., et al. "*Training a helpful and harmless assistant with reinforcement learning from human feedback*." *arXiv preprint arXiv:2204.05862*, 2022.

Bradley, R., and Terry, M. "*Rank Analysis of Incomplete Block Designs: I. The Method of Paired Comparisons*." *Biometrika*, 1952.

Christiano, P., Leike, J., Brown, T., Martic, M., Legg, S., et al. "*Deep Reinforcement Learning from Human Preferences*." *Advances in Neural Information Processing Systems*, 2017. [link]

Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., et al. "*Training Verifiers to Solve Math Word Problems*." *arXiv preprint arXiv:2110.14168*, 2021.

Kumar, A., He, Y., Markosyan, A., Chern, B., and Arrieta-Ibarra, I. "*Detecting Prefix Bias in LLM-based Reward Models*." *ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, 2025.

Lambert, N., Pyatkin, V., Morrison, J., Miranda, L., Lin, B., et al. "*RewardBench: Evaluating reward models for language modeling*." *Conference of the North American Chapter of the Association for Computational Linguistics*, 2025.

Lightman, H., Kosaraju, V., Burda, Y., Edwards, H., Baker, B., et al. "*Let's verify step by step*." *International Conference on Learning Representations*, 2024.

References (2/3)

Luce, R. "*Individual Choice Behavior: A Theoretical Analysis*." *Wiley*, 1959. [link]

Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., et al. "*Training language models to follow instructions with human feedback*." *Advances in Neural Information Processing Systems*, 2022. [link]

Plackett, R. "*The analysis of permutations*." *Journal of the Royal Statistical Society: Series C (Applied Statistics)*, 1975. [link]

Sharma, M., Tong, M., Korbak, T., Duvenaud, D., Askell, A., et al. "*Towards Understanding Sycophancy in Language Models*." *The Twelfth International Conference on Learning Representations*, 2024. [link]

Singhal, P., Goyal, T., Xu, J., and Durrett, G. "*A long way to go: Investigating length correlations in rlhf*." *arXiv preprint arXiv:2310.03716*, 2023.

Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., et al. "*Llama 2: Open foundation and fine-tuned chat models*." *arXiv preprint arXiv:2307.09288*, 2023.

Truong, S., Haupt, A., and Koyejo, S. "*Machine Learning from Human Preferences*." *Stanford University*, 2025. [link]

Von Neumann, J., and Morgenstern, O. "*Theory of games and economic behavior, 2nd rev*." *Princeton university press*, 1947.

Zhang, X., Xiong, W., Chen, L., Zhou, T., Huang, H., et al. "*From lists to emojis: How format bias affects model alignment*." *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025.

References (3/3)

Zheng, L., Chiang, W., Sheng, Y., Zhuang, S., Wu, Z., et al. "*Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*." *Advances in Neural Information Processing Systems*, 2023.

Zhu, B., Jordan, M., and Jiao, J. "*Principled reinforcement learning with human feedback from pairwise or k-wise comparisons*." *International Conference on Machine Learning*, 2023. [link]

Zhu, B., Frick, E., Wu, T., Zhu, H., Ganesan, K., et al. "*Starling-7B: Improving helpfulness and harmlessness with RLAIF*." *First Conference on Language Modeling*, 2024.